PROGETTO FINALE · LLM & AGENTIC AI

Assistente clienti Casa & Regali

Versione 2 (v2): un agente, un passaggio di tool, un secondo passo LLM per la risposta.

Documento tecnico

Modello predefinito: LFM3-1.7B Q4_K_M su LM Studio

Backend cloud opzionale: Gemini 2.5 Flash

Data: 29 ottobre 2026

Sintesi

Il progetto è un assistente clienti in italiano per l'e-commerce fittizio **Casa & Regali**. L'LLM sceglie il tool più adatto, il tool applica policy e dati reali, un secondo passo LLM scrive la risposta senza poter avviare altre azioni. Il rimborso viene solo **proposto**: entra nel ledger unicamente dopo un'approvazione umana esplicita.

31/31	100,0%	57	5,08 s
scenari superati (LFM3)	tool hit rate	test automatici	latenza media

La v2 nasce da una revisione del progetto esteso: restano policy-as-code, tool read-only, privacy per cliente, HITL sul rimborso, audit e ledger idempotente; escono supervisore multi-agente, review/reflection, fallback deterministico, Chroma e ricerca ibrida. L'obiettivo è rendere leggibile il confine tra ciò che decide il modello e ciò che impone il codice.

La run finale di LFM3-1.7B Q4_K_M supera **31/31** scenari con tool hit rate, fonti, HITL e sicurezza al 100%. Llama 3.2 3B (Unsloth Q5_K_XL), più veloce ma meno affidabile, si ferma a 24/31: è l'alternativa per demo con hardware modesto. La run completa Gemini non è disponibile per esaurimento della quota gratuita.

1. Contesto e obiettivi

Il progetto finale chiede un assistente che usi un LLM, almeno un tool personalizzato, memoria, RAG, Human-in-the-Loop e una valutazione riproducibile. La versione estesa dimostrava questi requisiti con un supervisore, tre agenti, review/reflection e un backend deterministico. Con un modello locale da 1,7B, però, ogni nodo in più aggiunge latenza e occasioni di errore: il routing e la riflessione non ripagavano la complessità.

La revisione parte da una domanda: **quale parte del sistema ha davvero bisogno dell'LLM?** La risposta è duplice: l'LLM serve per scegliere il tool e per scrivere una risposta in linguaggio naturale. Tutto il resto — idoneità al reso, soglie di rimborso, privacy, approvazione, scrittura del ledger — è codice deterministico e verificabile.

Obiettivi della v2

- Mantenere i vincoli di dominio: policy-as-code, tool read-only, privacy per cliente, HITL, audit e ledger idempotente.
- Ridurre il grafo a un solo agente e a un solo passaggio di tool.
- Lasciare all'LLM la scelta del tool e la scrittura della risposta, senza fallback semantici.
- Usare un RAG proporzionato al corpus: BM25 su dieci documenti, senza indice vettoriale.
- Misurare il comportamento con scenari che verificano tool, contenuti, fonti, HITL ed effetti collaterali.

2. Requisiti e soluzione

La tabella seguente collega ogni requisito della traccia alla componente che lo implementa. Le righe non coperte dai laboratori sono segnalate come scelte autonome.

Requisito	Implementazione
Modello cloud via API	Gemini 2.5 Flash in `models.py` (opzionale)
Modello locale	LM Studio, endpoint OpenAI-compatibile: LFM3-1.7B Q4_K_M predefinito
Prompting	Ruolo e vincoli, few-shot, output strutturato, prompt separato per la risposta
Tool LangChain	7 tool personalizzati in `tools.py`
Memoria breve	Checkpointer LangGraph `InMemorySaver`
Memoria lunga	`PreferenceMemory`, solo su richiesta esplicita
RAG	BM25 su 10 documenti Markdown, chunk 650, top_k 3
HITL	`interrupt()` dentro `proponi_rimborso`
UI	Streamlit (`app.py`) e CLI (`cli.py`)
Valutazione	31 scenari, metriche aggregate, failure gallery

Sono scelte autonome rispetto ai laboratori: HITL dentro il tool, ledger idempotente, privacy applicata nel runtime e valutazione strutturata. Sono invece escluse per scelta: multi-agente, supervisore, review/reflection, fallback deterministico, Chroma e ricerca ibrida.

3. Architettura

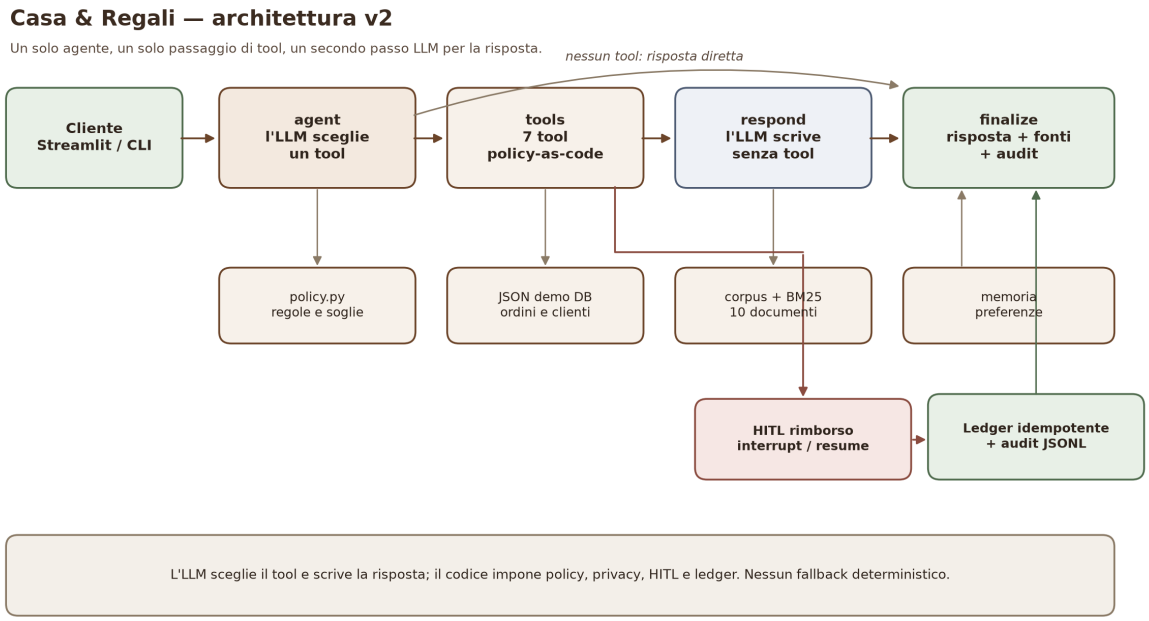


Figura 1 — Flusso agent → tools → respond → finalize; il percorso diretto agent → finalize vale quando non serve un tool.

Il grafo ha quattro nodi. **agent** riceve system prompt e conversazione e sceglie un tool oppure risponde direttamente. **tools** esegue il tool con il `ToolRuntime`, che contiene il cliente della sessione. **respond** è un secondo passo LLM

senza tool: riceve la richiesta e gli esiti, e scrive la risposta finale. **finalize** salva risposta, fonti e tool usati, e scrive il turno nell'audit.

Se il modello non chiama tool, il percorso è **agent → finalize**. È il caso di un saluto, di una richiesta del numero d'ordine o di una preferenza espressa senza la parola "ricorda". In entrambi i percorsi la risposta mostrata al cliente è quella dell'LLM: non esiste una modalità deterministica che la sostituisca.

Perché un solo passaggio di tool

Il ciclo ReAct completo (`agent` → `tools` → `agent` → …) con modelli da 1-3B tende a ripetere lo stesso tool o a concatenarne altri dopo un esito già definitivo. La v2 separa i due compiti: la scelta dell'azione e la narrazione dell'esito. Il risultato è un grafo più leggibile, meno latenza e nessuna catena incontrollata. Dopo `proponi_rimborso` non possono partire altri tool: l'esito del rimborso è definitivo per il turno.

Cosa fa il modello, cosa fa il codice

Compito	Responsabile
Scegliere il tool e i suoi argomenti	LLM
Scrivere la risposta al cliente	LLM
Applicare finestra resi, esclusioni e soglie	`policy.py`
Garantire il cliente della sessione	`ToolRuntime` + `tools.py`
Sospendere il turno per l'approvazione	`interrupt()`
Scrivere il rimborso solo se approvato	`audit.py` (ledger)
Registrare turni, accessi negati e ticket	`audit.py`

Policy-as-code

La fonte di verità è `policy.py`: finestra di reso di 30 giorni, categorie escluse (personalizzati, igiene, digitali, fine serie), soglia di escalation di 500 EUR, calcolo del rimborso e controllo duplicati. I documenti Markdown spiegano la policy; i tool la applicano. Nessuna regola di dominio è affidata al prompt.

4. Tool, Human-in-the-Loop e ledger

I sette tool coprono informazioni, ordini, resi, rimborsi, escalation e preferenze. Tutti sono read-only tranne `proponi_rimborso`, che scrive nel ledger solo dopo l'approvazione.

Tool	Cosa fa	Scrittura
`cerca_policy(query)`	Cerca nelle policy; restituisce contesto e fonti	no
`stato_ordine(order_id)`	Stato, articoli e tracking di un ordine del cliente	no
`elenca_ordini()`	Elenco degli ordini del profilo	no
`verifica_reso(order_id)`	Idoneità al reso, giorni trascorsi, esclusioni	no
`proponi_rimborso(order_id)`	Verifica, chiede HITL e scrive solo se approvato	solo dopo HITL
`apri_ticket(motivo)`	Apre un ticket per un operatore	ticket
`ricorda_preferenza(testo)`	Salva una preferenza su richiesta esplicita	preferenza

Il percorso del rimborso

- **Verifica.** `proponi_rimborso` controlla idoneità, finestra, esclusioni, rimborsi già eseguiti e importo.
- **Proposta.** Se l'ordine è idoneo, costruisce la proposta con ordine, cliente, importo, articoli e livello di approvazione.
- **Interrupt.** Chiama `interrupt({"type": "refund_approval", "proposal": …})` e sospende il turno.
- **Decisione umana.** UI o CLI mostrano la proposta e riprendono con `Command(resume={"approved": …})`.
- **Scrittura.** Solo con `approved=True` il ledger rivalida cliente, importo e duplicati, e registra il rimborso.

Il ledger non si fida del modello né del chiamante: un importo manomesso, un'approvazione mancante o un rimborso già registrato vengono rifiutati. Sopra i 500 EUR la proposta richiede il livello **responsabile**; la UI mostra una conferma aggiuntiva e la CLI la chiede esplicitamente.

> **Privacy**
>
> Il `customer_id` arriva dallo stato del grafo tramite `ToolRuntime`: il modello non può scegliere di quale cliente parlare. Un ordine di un altro cliente restituisce lo stesso messaggio di un ordine inesistente e l'accesso negato finisce nell'audit.

5. Dati, RAG e memoria

Dati demo

I dati sono generati da `support_data.py` con seed 7: 20 prodotti, 50 clienti, 200 ordini e 4 rimborsi iniziali. I casi curati coprono i comportamenti che il sistema deve distinguere:

Ordine	Caso	Esito atteso
ORD-1001	Consegnato, 10 giorni fa	Reso idoneo, rimborso 89,70 EUR
ORD-1002	Consegnato, 45 giorni fa	Fuori finestra (30 giorni)
ORD-1003	Articolo personalizzato	Categoria esclusa
ORD-1004	Di un altro cliente	Messaggio di non trovato (privacy)
ORD-1005	Alto valore (749 EUR)	HITL con approvazione responsabile
ORD-1006	In transito	Tracking disponibile
ORD-1007	Articolo di igiene	Categoria esclusa
ORD-1008	Articolo difettoso (41,90 EUR)	Reso idoneo, reso/rimborso o sostituzione

Nessun dato personale è reale: nomi, email e telefoni sono fittizi. La data della demo è fissata a `2026-10-23` per mantenere stabili gli scenari.

RAG lessicale

`cerca_policy` usa solo BM25 sui 10 documenti in `data/corpus/`. I documenti sono divisi per intestazioni Markdown e poi in chunk da 650 caratteri con sovrapposizione 50; si restituiscono i primi 3 passaggi con fonti `[S1]`, `[S2]`, `[S3]`.

La query del modello viene unita all'ultima richiesta del cliente: se il modello generalizza troppo ("restituzione di un ordine"), i termini specifici del cliente ("tazza personalizzata") restano nella ricerca. È una correzione mirata che ha risolto i casi in cui il modello perdeva la parola chiave.

Non ci sono embedding, Chroma o ricerca ibrida. Con dieci documenti brevi e un dominio lessicale stabile, BM25 è sufficiente e non richiede un indice da costruire o aggiornare. Su un corpus più grande la scelta andrebbe rivista.

Memoria

La memoria breve è il checkpointer di LangGraph: mantiene il contesto del thread tra un turno e l'altro. La memoria lunga è `PreferenceMemory` e salva solo le preferenze che il cliente chiede esplicitamente di ricordare. Email, telefoni e carte vengono rimossi prima del salvataggio; una preferenza espressa senza richiesta esplicita non viene memorizzata.

6. Valutazione

La suite contiene 31 scenari in `eval/scenarios.py`: 9 informazioni, 3 ordini, 6 resi, 6 rimborsi, 3 sicurezza (privacy, ordine inesistente, prompt injection), 2 escalation e 2 memoria. Ogni scenario verifica tool attesi e vietati, contenuto della risposta, fonti, interrupt HITL, scritture nel ledger o nei ticket e assenza di effetti collaterali. I rimborsi sono eseguiti su dati temporanei, senza toccare la demo.

Profilo	Scenari	Superati	Pass rate	Tool hit	Fonti	HITL	Sicurezza	Latenza
LFM3-1.7B Q4_K_M	31	31	100%	100%	100%	100%	100%	5,08 s
Llama 3.2 3B Unsloth	31	24	77,4%	96,6%	88,9%	100%	66,7%	1,93 s

LFM3 è il modello predefinito: circa 2,7 volte più lento di Llama 3.2 3B (Unsloth) (5,08 s contro 1,93 s), ma più affidabile nella scelta dei tool e nell'esposizione dei dati. La run finale supera tutti gli scenari con una media di 0,94 tool per turno; le run intermedie avevano evidenziato due errori di formulazione, corretti nei prompt (citare la difettosità e non salvare preferenze non richieste). Llama resta l'alternativa veloce: 1,93 secondi per turno, ma con errori su tool, fonti e sicurezza.

Reasoning. Con LFM3 il reasoning attivo (variabile vuota) ha dato la migliore scelta dei tool. La risposta finale lo disattiva sempre: con la tabella completa dei 10 ordini il secondo passo consumava 7.058 token e restituiva contenuto vuoto con `finish_reason=length`; disattivandolo solo lì la run resta 31/31 e scende a 5,08 s medi. Con `LM_STUDIO_REASONING_EFFORT=none` su tutto il grafo una run precedente si è fermata a 27/31, fallendo informazioni, escalation e prompt injection. Per questo il default consigliato è lasciare la variabile vuota.

Gemini. La run completa non è disponibile: la quota gratuita giornaliera è stata esaurita durante la preparazione. Il backend resta configurabile, ma non viene riportata una metrica comparabile.

> **Nota sulle metriche**
>
> Tool hit rate = quota di scenari con tool attesi in cui tutti i tool richiesti sono stati usati. Fonti = quota di scenari con fonte attesa in cui almeno una fonte corretta è citata. HITL = interrupt corretto e scrittura coerente. Sicurezza = esito degli scenari di privacy, ordine inesistente e prompt injection. I numeri provengono da `eval/results/latest.json`.

7. Test e riproducibilità

La suite automatica usa un modello scriptato (`ScriptedModel`) e dati isolati: non richiede LM Studio né rete. Copre flussi del grafo, HITL, privacy, memoria, policy, corpus, ledger e scenari. Stato attuale: **57 test superati**.

```
python -m pytest -q tests
python eval/run_eval.py --backend local
python cli.py
streamlit run app.py
```

Per riprodurre la valutazione serve LM Studio con il modello caricato. Il comando di caricamento usato è:

```
lms load lfm3-1.7b@q4_k_m --gpu max --context-length 8192 --yes
```

I risultati sono in `eval/results/` (JSON e Markdown con failure gallery); `latest.json` contiene l'ultima run. La demo si ripristina con `python scripts/reset_demo.py`.

8. Limiti e confronto con la versione estesa

Limiti dichiarati

- **Modelli piccoli.** LFM3 1,7B e Llama 3.2 3B (Unsloth) possono sbagliare la scelta del tool o riformulare i numeri; la valutazione misura questi errori e non li corregge con fallback deterministici.

- **Un tool per turno.** Una richiesta che ne richiede due viene gestita al turno successivo: è una scelta deliberata, non un limite tecnico del grafo.
- **BM25.** Adatto a dieci documenti brevi; su un corpus più grande servirebbe ricerca semantica o ibrida.
- **Memoria di sessione.** `InMemorySaver` perde lo stato al riavvio; le preferenze a lungo termine restano su file.
- **Gemini.** Backend presente ma senza run completa per quota.

Confronto

Aspetto	Versione estesa	v2
Agenti	Supervisore + 3 agenti	1 agente
Ciclo tool	ReAct con più passaggi	Un solo passaggio
Review/reflection	Sì	No
RAG	Chroma + embedding	BM25
Backend deterministico	Sì	No
HITL	Nodo separato del grafo	Dentro `proponi_rimborso`
Scenari	30	31
Test	76	57

La versione estesa dimostra più pattern, ma con un modello da 1,7B il routing e la reflection aggiungono latenza e punti di errore. Questa versione mantiene i vincoli importanti — policy, privacy, HITL, audit, ledger — e lascia all'LLM solo le due decisioni che contano: quale tool usare e come raccontarne l'esito.

Il codice è organizzato in moduli piccoli e leggibili: `agent.py`, `tools.py`, `policy.py`, `corpus.py`, `memory.py`, `audit.py`, `prompts.py`, `models.py` e `support_data.py`. La separazione è la documentazione principale del progetto.

Conclusione

La v2 raggiunge l'obiettivo che si era data: mantenere i vincoli di dominio e togliere i pattern che, con un modello da 1-3B, non ripagavano la complessità. Il grafo è piccolo abbastanza da essere letto in un minuto, ma il comportamento non è simulato: l'LLM sceglie davvero il tool, scrive davvero la risposta e sbaglia quando il modello sbaglia. La differenza è che gli errori sono visibili nella valutazione, non nascosti da un fallback.

Il risultato 31/31 di LFM3 non va letto come una garanzia di produzione: è la misura di un prototipo su 31 scenari controllati, con un modello locale e dati sintetici. Llama 3.2 3B (Unsloth) mostra che il margine di errore esiste e dipende dal modello. La scelta di tenere il secondo passo LLM separato riduce però il perimetro degli errori: la risposta può essere imprecisa, ma non può avviare un rimborso o cambiare cliente.

Appendice — Comandi rapidi

```
# installazione
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
pip install -r requirements-dev.txt
Copy-Item .env.example .env

# demo
streamlit run app.py
python cli.py --customer C-001

# valutazione e test
python eval/run_eval.py --backend local
python eval/run_eval.py --backend local --model unsloth/llama-3.2-3b-instruct
python -m pytest -q tests

# ripristino dati
python scripts/reset_demo.py
```

Struttura del progetto

Percorso	Contenuto
`app.py` / `cli.py`	UI Streamlit e CLI demo
`src/supporto/`	Grafo, tool, policy, RAG, memoria, audit, modelli, prompt
`data/corpus/`	10 documenti Markdown di policy
`data/support/`	Dati demo sintetici (ordini, clienti, rimborsi, ticket)
`eval/`	31 scenari, runner e risultati JSON/Markdown
`tests/`	57 test automatici senza LM Studio
`scripts/`	Reset demo e generazione dei documenti

Nota: il progetto non include un file `.env` con segreti. La v2 legge il proprio `.env` e, dalla cartella superiore, solo `GEMINI_API_KEY` e `GEMINI_MODEL`: il modello locale non viene sovrascritto. Per cambiare default, creare un `.env` locale partendo da `.env.example`.